VELOOP REWARDS - Data Quality Control Framework

Data Analyst Internship, Task 01 | Work Package L

Authenticity notice: Every rule below is **Proposed**. No tolerance, threshold or limit is invented (they are marked 'to be agreed'), and no simulated or real VELOOP statistics appear here.

1. Summary

- **39 quality rules** (11 Critical) placed across six layers: Application (10), Data Collection (10), Raw Data (3), Transformation (11), Analytics Layer (4), Dashboard (1) (counted by primary layer).
- **All 14 issue types in the brief are covered** (duplicate IDs to impossible state transitions), plus cross-table consistency, timeliness, completeness and separation of simulated from production data. The spreadsheet checks this by formula.
- **Every rule has a primary layer and a backstop layer**, an action on failure, a severity and an owner. The brief's five examples (ID validation, allowed status, duplicate transaction, timestamp, referential integrity) are DQ001, DQ009, DQ018, DQ005 and DQ006, DQ016.
- **Rule IDs will be reused in the Data Quality Audit** (Work Package K): each issue found in Messy_Data will name the rule that detects it.

2. Design principles

- Prevent first: the earliest sensible layer should stop a problem, because fixing at the dashboard is the most expensive place.
- Never silently change meaning: only formatting is auto-corrected; IDs, amounts, units, statuses and timestamps are never guessed.
- Keep raw data untouched, so every correction can be traced and reversed.
- Quarantine, do not delete: every held record keeps its rule ID and reason.
- Measure quality as numbers, not feelings: the metrics in Section 5 are shown beside the KPIs.
- Start small: scheduled SQL checks and a quarantine table are enough at first; add tooling only when volume justifies it.

3. Where each control operates

Application, Data Collection, Raw Data, Transformation, Analytics Layer, Dashboard. A problem is stopped at the earliest sensible layer; later layers are backstops.

Layer	Purpose	Typical Controls	Failure Handling	Owner
Application	Prevent bad data from being created in the first place	Database constraints (unique, not null, enum); input validation; server-generated IDs and timestamps; idempotent writes	Reject the request with a clear error; log it	Engineering
Data Collection	Check each event as it arrives	ID and timestamp checks; schema and event-name check against the catalogue; required properties; de-duplication by event_id	Quarantine the bad event with a reason; accept the rest	Engineering
Raw Data	Keep exactly what arrived, unchanged	Append-only storage; add received_timestamp; volume and freshness monitoring only	Never edit; alert on gaps or delays	Engineering / Data
Transformation	Standardise and validate against the dictionary	Trim and case standardisation; type checks; reference-table checks; foreign keys; state transitions; produce a clean table and a quarantine table	Auto-correct only safe formatting and log it; quarantine or flag the rest	Data / Analytics
Analytics Layer	Check that tables agree with each other	Reward-wallet-withdrawal reconciliation; balance checks; build the quality metrics from clean records only	Alert owners; keep failing records out of KPIs or label them	Data / Analytics
Dashboard	Show the user how far to trust the numbers	'Data as of' time; count of excluded or quarantined records; block unit mixing; show quality status beside key figures	Block unsafe calculations; display warnings	Analytics / Product

4. Quality rule catalogue

Rule ID	Rule	Issue Type	Applies To	Rule Logic	Primary Layer	Backstop Layer	Action on Failure	Severity	Owner	Status
DQ001	Identifier present	missing IDs	All primary keys; user_id and other foreign keys	Not null, not empty, not a placeholder such as 'null' or 'N/A'	Application	Data Collection; Raw Data	Reject at source (app); Quarantine (collection)	Critical	Engineering	Proposed
DQ002	Primary key unique	duplicate IDs	Every entity's primary key	Exactly one row per primary key (user_id, transaction_id, withdrawal_id, reward_id...)	Application	Transformation	Reject at source (database constraint); Quarantine later duplicates	Critical	Engineering	Proposed
DQ003	Identifier format	missing IDs	All ID fields	Matches the agreed pattern; no spaces; no personal data inside IDs	Data Collection	Raw Data	Quarantine	High	Engineering	Proposed
DQ004	Timestamp present	missing timestamps	event_timestamp, created_at, requested_at, started_at	Not null for every record where the dictionary says Required	Application	Data Collection	Reject at source (app); Quarantine (collection)	High	Engineering	Proposed
DQ005	Timestamp format	malformed dates	All timestamp fields	Parses as ISO 8601; day/month order is never guessed	Data Collection	Transformation	Quarantine	High	Engineering	Proposed
DQ006	Timezone explicit	malformed dates	All timestamp fields	Carries an explicit offset or is already UTC; stored in UTC (A012)	Data Collection	Transformation	Quarantine if the offset is missing	High	Engineering	Proposed
DQ007	No future timestamps	malformed dates	event_timestamp and business timestamps	Not later than received_timestamp plus an agreed clock tolerance	Data Collection	Transformation	Flag and load	Medium	Engineering	Proposed
DQ008	Timestamp order	impossible state transitions	Task_Activity, Withdrawals, Referrals, Users	started_at <= completed_at; requested_at <= processing_at <= final_at; created_at <= registered_at <= activated_at <= converted_at	Transformation	Analytics Layer	Flag and load	Medium	Data / Analytics	Proposed
DQ009	Allowed status values	invalid statuses	status, validation_status, account_status, reward_status, ticket_status	Value is in the approved list held in a reference table	Application	Data Collection; Transformation	Reject at source (app); Quarantine (collection)	High	Engineering	Proposed
DQ010	Capitalization standard	inconsistent capitalization	All enum and category fields	Stored in lowercase snake_case; 'Completed' and 'COMPLETED' become 'completed'	Transformation	Data Collection	Auto-correct and log	Low	Data / Analytics	Proposed
DQ011	No stray whitespace	trailing spaces	All string fields	Value equals its trimmed form	Transformation	Data Collection	Auto-correct and log	Low	Data / Analytics	Proposed
DQ012	Numeric value valid	invalid numeric values	amount, reward_amount, requested_amount, balance_after	Parses as decimal; no text, no NaN, no thousands separators	Application	Data Collection; Transformation	Reject at source (app); Quarantine (collection)	High	Engineering	Proposed
DQ013	Amounts are positive	negative values where inappropriate	amount, reward_amount, requested_amount, completed_amount	Greater than 0; direction and reversal entries carry the sign, not the amount	Application	Transformation	Reject at source (app); Quarantine (collection)	Critical	Engineering	Proposed
DQ014	Unit present and approved	invalid statuses	unit, reward_unit, requested_unit	Present and in the approved unit list; one unit per entry	Application	Data Collection	Reject at source	Critical	Engineering	Proposed
DQ015	Conditional fields present	unexpected nulls	Fields marked Conditional in the dictionary	completed_at when status = completed; reason_category when rejected or reversed; related_transaction_id on reversals	Data Collection	Transformation	Quarantine or Flag and load	High	Engineering	Proposed
DQ016	User exists	orphan records	Every user_id foreign key	Value exists in Users	Transformation	Raw Data (kept as received); Analytics Layer	Quarantine (held, not deleted)	Critical	Data / Analytics	Proposed
DQ017	Other foreign keys exist	orphan records	task_id, task_activity_id, reward_id, wallet_transaction_id, related_transaction_id, referral foreign keys	Value exists in its parent table	Transformation	Analytics Layer	Flag and load	High	Data / Analytics	Proposed
DQ018	Duplicate wallet transaction	duplicate IDs	Wallet_Transactions	One row per transaction_id and one per idempotency_key	Application	Raw Data; Transformation	Reject at source (repeat request returns the original entry)	Critical	Engineering	Proposed
DQ019	Duplicate event	duplicate events	Feature_Events	One row per event_id; a retry reuses the same event_id	Data Collection	Raw Data; Transformation	Keep first, count and log the rest	High	Engineering	Proposed

Rule ID	Rule	Issue Type	Applies To	Rule Logic	Primary Layer	Backstop Layer	Action on Failure	Severity	Owner	Status
DQ020	Event name in catalogue	inconsistent naming	event_name	In the Event Tracking Plan; lowercase object_action	Data Collection	Transformation	Quarantine	High	Engineering	Proposed
DQ021	Field names follow dictionary	inconsistent naming	All tables	Column names and types match the data dictionary exactly	Transformation	Analytics Layer	Fail the load; Alert	Medium	Data / Analytics	Proposed
DQ022	Event properties complete	unexpected nulls	Feature_Events.properties	All required properties for the event are present with the right type	Data Collection	Transformation	Quarantine	High	Engineering	Proposed
DQ023	Schema version known	inconsistent naming	schema_version	Version exists in the catalogue	Data Collection	Raw Data	Quarantine	Medium	Engineering	Proposed
DQ024	Task state transitions	impossible state transitions	Task_Activity.status	Only allowed moves, for example no validated after failed	Transformation	Analytics Layer	Flag and load	High	Data / Analytics	Proposed
DQ025	Reward state transitions	impossible state transitions	Rewards.reward_status	Only allowed moves; no reversed to credited, no credited to pending	Transformation	Analytics Layer	Flag and load	High	Data / Analytics	Proposed
DQ026	Withdrawal state transitions	impossible state transitions	Withdrawals.status	Only allowed moves; no completed to processing, no completed to rejected	Transformation	Analytics Layer	Flag and load; Alert	Critical	Data / Analytics	Proposed
DQ027	Referral stage order	impossible state transitions	Referrals.status	No activation before registration; no conversion before activation unless the rule allows	Transformation	Analytics Layer	Flag and load	Medium	Data / Analytics	Proposed
DQ028	Reversal link valid	orphan records	related_transaction_id, original_reward_id	Exists, is not itself, and does not point to another reversal	Application	Transformation	Reject at source (app); Flag and load	High	Engineering	Proposed
DQ029	Self-referral	impossible state transitions	Referrals	referred_user_id differs from referring_user_id	Application	Transformation	Reject at source (app); Flag and load	High	Engineering	Proposed
DQ030	Reward-wallet agreement	Cross-table consistency	Rewards, Wallet_Transactions	Each credited reward has exactly one reward_credit entry with reference_id = reward_id	Analytics Layer	Dashboard	Alert	Critical	Data / Analytics	Proposed
DQ031	Wallet balance agreement	Cross-table consistency	Wallet_Transactions	Sum of completed credits minus debits equals the reported balance per user and unit	Analytics Layer	Dashboard	Alert	Critical	Data / Analytics	Proposed
DQ032	Withdrawal-wallet agreement	Cross-table consistency	Withdrawals, Wallet_Transactions	Each completed withdrawal has exactly one active withdrawal_debit entry	Analytics Layer	Dashboard	Alert	Critical	Data / Analytics	Proposed
DQ033	Exchange pairing	Cross-table consistency	Wallet_Transactions	Each exchange_group_id has one debit and one credit	Transformation	Analytics Layer	Flag and load	High	Data / Analytics	Proposed
DQ034	Task-reward link agreement	Cross-table consistency	Task_Activity.reward_id, Rewards.task_activity_id	Both links agree wherever both exist	Analytics Layer	Dashboard	Flag and load	High	Data / Analytics	Proposed
DQ035	No mixed-unit totals	Cross-table consistency	Dashboard calculations	No sum or comparison across units without an agreed conversion rule (A002)	Dashboard	Analytics Layer	Block	High	Analytics / Product	Proposed
DQ036	Data freshness	Timeliness	Raw tables	Latest received_timestamp is within the agreed age; dashboards show 'data as of'	Raw Data	Dashboard	Alert	Medium	Engineering	Proposed
DQ037	Late-arriving data	Timeliness	Feature_Events	Delay between event_timestamp and received_timestamp is inside the agreed tolerance; later arrivals trigger a restatement of the affected period	Data Collection	Analytics Layer	Flag and load	Medium	Engineering	Proposed
DQ038	Volume anomaly	Completeness	Raw tables	Daily row counts stay inside an agreed band around the recent average	Raw Data	Dashboard	Alert	Medium	Engineering	Proposed
DQ039	Simulated and production data separated	Environment separation	All datasets	Simulated data lives in a separate, labelled location and is never loaded into production tables; dashboards read only from the approved source	Raw Data	Analytics Layer	Block	Critical	Data / Analytics	Proposed

5. Failure handling

Action	What It Does	Used For	Never Used For
Reject at source	The write or request fails and the caller sees why	Missing keys, invalid enums, non-positive amounts, repeated transactions	Silent dropping of data
Quarantine	The record is held in a quarantine table with the rule ID and reason; it is not used in KPIs until resolved	Malformed events, orphans, unknown event names, unreadable dates	Deleting the record
Flag and load	The record loads with a quality flag so reports can include or exclude it	Order problems, impossible transitions, soft inconsistencies	Hiding the flag from reports
Auto-correct and log	A deterministic formatting fix is applied and logged; the raw value is kept	Trailing spaces, capitalization	Changing IDs, amounts, units, statuses or timestamps
Alert	The owner is notified and the issue is logged	Reconciliation breaks, freshness, volume anomalies	Relying on someone noticing a dashboard
Block	The unsafe action is not allowed	Summing across units, mixing simulated and production data	-

Quality metrics

These are shown beside the KPIs. No target values are set.

Quality Metric	Definition	Formula	Applies To	Status
Completeness rate	Share of required fields that are filled	Non-null required values / required values	Every table	Proposed
Validity rate	Share of values that pass type and allowed-value checks	Values passing DQ009, DQ012, DQ013, DQ014 / values checked	Every table	Proposed
Duplicate rate	Share of rows or events that repeat a key	Duplicate rows / total rows, by primary key or event_id	Every table, Feature_Events	Proposed
Orphan rate	Share of foreign key values with no parent	Orphan rows / rows with that foreign key, per foreign key	Every foreign key	Proposed
Quarantine rate	Share of arriving records held back	Quarantined records / records received, by rule ID	Collection, Transformation	Proposed
Auto-correction rate	Share of records changed by safe formatting fixes	Auto-corrected values / values checked	Transformation	Proposed
Reconciliation exceptions	Cases where tables disagree	Count of failures of DQ030 to DQ032 (see KPI T01)	Rewards, Wallet, Withdrawals	Proposed
Unresolved soft-reference rate	Share of event or reference keys that do not resolve to a record	Unresolved keys / keys present	Feature_Events, Wallet_Transactions.reference_id	Proposed
Event delivery delay	Typical lag between event and receipt	Median and 90th percentile of received_timestamp - event_timestamp (KPI T03)	Feature_Events	Proposed
Open issues by severity	Backlog of unresolved quality issues	Count of open issues grouped by severity	All	Proposed

6. Quarantine table (proposed structure)

Field	Type	Meaning
quarantine_id	string	Key of the quarantined record
source_table	string	Table or event stream the record came from
source_record_id	string	ID of the record, if it had one
rule_id	string	Quality rule that caught it (DQ###)
reason	string	Short reason, no personal data
detected_at	timestamp (UTC)	When it was caught
raw_reference	string	Pointer to the untouched raw record
status	string	open, resolved, discarded with approval
resolved_at	timestamp (UTC)	When it was resolved

7. Severity scale and issue lifecycle

Severity	Meaning	Typical Response
Critical	Affects identity, balances, payouts or reconciliation; reporting built on it cannot be trusted	Alert the owner at once; do not publish affected figures
High	Breaks a KPI or a major join	Fix in the current work cycle; flag affected reports
Medium	Distorts a segment or delays data	Fix in planned work; note on the report
Low	Formatting or cosmetic; safe to standardise	Auto-correct and log

Issue lifecycle: Detected (rule ID, count and sample recorded) > Triaged (severity and owner assigned) > In progress > Resolved (correction applied or approved to discard) > Verified (rule re-run and passes) > Prevented (a control is added at the earliest sensible layer)

8. Practical scope

- Start with scheduled SQL checks, a quarantine table and a data-as-of banner. These need no new platform.
- Add monitoring tools or a streaming layer only when volume or the number of sources makes the simple approach unreliable.

9. Open questions

- What clock tolerance and freshness limits apply? (A012; no value is set here)
- Where do the approved-value reference tables live, and who maintains them?
- Who triages the quarantine table and how often?
- What volume band should trigger a volume alert?
- Which dashboard tool will show data-as-of and quality status (kept simple at first)?

The supplied dataset was used to test these rules (Deliverable 13). It covers the Users entity only, so rules that need other datasets could not be tested. The audit proposes three further rules, DQ040 to DQ042, to be added through the change log.